

Blockchain-Based Decentralized Federated Learning With On-Chain Model Aggregation and Incentive Mechanism for Industrial IoT

QING YANG¹ (Member, IEEE), WEI XU² (Student Member, IEEE), TAOTAO WANG¹ (Member, IEEE),
HAO WANG^{3,4} (Member, IEEE), XIAOXIAO WU¹ (Senior Member, IEEE),
BIN CAO⁵ (Senior Member, IEEE), AND SHENGLI ZHANG¹ (Senior Member, IEEE)

¹College of Electronics and Information Engineering, Shenzhen University, Shenzhen 518060, China

²Computer Science Department, New York University, New York, NY 10012, USA

³Department of Data Science and Artificial Intelligence, Faculty of Information Technology, Monash University, Melbourne, VIC 3800, Australia

⁴Monash Energy Institute, Monash University, Melbourne, VIC 3800, Australia

⁵State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing 100876, China

CORRESPONDING AUTHOR: T. WANG (e-mail: ttwang@szu.edu.cn)

This work was supported in part by the National Key Research and Development Program of China under Grant 2021YFB1714100; in part by the National Natural Science Foundation of China under Grant 62372309; in part by the Guangdong Basic and Applied Basic Research Foundation under Grant 2024A1515012407; in part by the Shenzhen Science and Technology Program under Grant JCYJ20220531101015033; and in part by the Zhejiang Lab Open Research Project under Grant K2022PD0AB04.

(Qing Yang and Wei Xu are co-first authors.)

ABSTRACT Federated learning (FL) is an emerging machine learning paradigm that enables the participants to train a global model without sharing the training data. Recently, FL has been widely deployed in industrial IoT scenarios because of its data privacy and scalability. However, the current FL architecture relies on a central server to orchestrate the FL process, thus incurring a risk of privacy leakage and single-point failure. To address this issue, we propose a fully decentralized FL architecture based on blockchain technology. Unlike existing blockchain-based FL systems that use blockchain for coordination or storage, we use blockchain as a trustable computing platform for model aggregation. Furthermore, we model the interaction between the FL task publisher and participants as a Stackelberg game and design a rewarding mechanism to incentivize participants to contribute to the FL task. We build a prototype system of the proposed decentralized FL architecture and implement an FL-based damaged package detection application to evaluate the proposed approach. Experimental results show that the blockchain-based decentralized FL is feasible in a practical industrial IoT scenario, and the incentive mechanism performs well with real application data.

INDEX TERMS Decentralized federated learning, blockchain, incentive mechanism, industrial IoT.

I. INTRODUCTION

MACHINE learning (ML) is playing an increasingly significant role in industrial IoT applications, such as smart manufacturing, smart healthcare, smart transportation, and smart factory [1], [2]. Conventional ML methods rely on a large amount of centrally-stored data collected from massive industrial applications to train the model [3]. However, collecting data directly from the users is infeasible in many industrial scenarios (e.g., electric vehicles [4])

because these data are privacy-sensitive and legally protected by laws, such as the general data protection regulation (GDPR) [5]. Recently, federated learning (FL) [6] has emerged as a promising solution to this problem by collaboratively utilizing decentralized data without explicitly exchanging them.

In the conventional FL system, a central coordinator orchestrates multiple participants to learn a model with their private data collaboratively. In each learning round, the

participant downloads the current global model, improves it by learning its own dataset, and updates the improved model to the coordinator. Then, the improved models from all the participants are integrated by the coordinator to generate an improved global model for the next round of learning. The FL can achieve lower latency and power consumption than the centralized ML while preserving participants' private data. As a result, FL has been applied in various practical projects (e.g., Google's Gboard) and adopted by mainstream machine learning platforms, such as Tensorflow.

However, several challenges limit the application of FL in practical industrial scenarios. First, existing FL systems rely on a powerful central coordinator and complicated mechanisms to orchestrate the participants, resulting in a high operational cost and entry barrier [7]. Second, the operation of the coordinator in current FL systems is centralized and opaque to the participants. The "black box" effect of the coordinator impedes the trust and scalability of the FL system. Third, the existing FL platform lacks a fair and efficient incentive mechanism for the participants. Individual data owners are unmotivated to participate in FL unless they receive reasonable rewards proportional to the contributions.

To address the above challenges, we employ the *blockchain technology* to develop a blockchain-based decentralized FL system for industrial applications. Blockchain, as the supporting technology of cryptocurrency and decentralized applications (DApps), provides a new angle to reform the FL architecture [8]. Blockchain implements a decentralized database on multiple nodes by synchronizing their states with the consensus algorithm [9], [10], and provides a trustable computing platform by allowing users to execute generic computer programs as smart contracts [11]. We propose using blockchain as a trustable computing platform for on-chain model aggregation and design an incentive mechanism based on a Stackelberg game to motivate the participants. The main contributions of this work are as follows.

- 1) We propose and implement a fully decentralized federated learning platform based on blockchain technology. Our work utilizes the blockchain as a trustable computing platform for the on-chain model aggregation and incentive mechanism.
- 2) We design an incentive mechanism based on a two-stage Stackelberg game between the FL task publisher and participants. The incentive mechanism provides rewards to incentivize participants to optimize the training time and cost. We develop a customized algorithm to solve the optimal reward utilizing the unique structure of the two-stage problem, which is often nonconvex and challenging to solve.
- 3) We build a practical prototype system of the proposed blockchain-based decentralized FL architecture. Further, we evaluate the prototype system with a

typical industrial IoT application of damaged package detection.

The remainder of the paper is organized as follows. Section II reviews the background and related works. Section III describes the system model of the blockchain-based decentralized federated learning architecture. Section IV presents the design of the incentive mechanism. In Section V, we implement and evaluate the proposed FL architecture with extensive experiments. Section VI concludes this work.

II. RELATED WORK

This section reviews the related works from three aspects: design and implementation of the state-of-the-art FL architectures, blockchain technology and its application in FL systems, and incentive mechanism and reward allocation for FL applications.

After McMahan et al. first proposed the concept of federated learning and implemented the Federated Averaging (FedAvg) algorithm for model aggregation [12], FL sparked wide research interests in the field of machine learning. Geyer et al. [13] employed the stochastic gradient descent method to implement the differential privacy protection for the client side in FL. In [13], the authors randomly subsampled and distorted the training dataset and then added Gaussian noises into the trained model before sending the data to a central server. This method could protect the clients' privacy by preventing malicious participants from inferring other participants' private information from the model parameters. However, this method relies on a trustable central server, which is usually infeasible in practice, thus incurs the risk of single-point failure. To address this challenge, Bonawitz et al. proposed a secure model aggregation method based on secure multiparty computation [14]. This method allows the central server to only learn the participants' information after the aggregation. In [15], the authors employed additive homomorphic encryption and differential privacy to prevent the honest-but-curious central server from exploiting the clients' local models. The above research works improved the feasibility and privacy of FL, but still assumed an honest central server for model aggregation. Also, differential privacy and homomorphic encryption incur heavy overhead in the FL system. In this work, we propose a novel decentralized FL architecture that leverages the blockchain platform for model aggregation, thus removing the need for a trustable central server.

Recently, the emergence of blockchain technology has inspired several new research directions that apply blockchain in FL scenarios. Blockchain is a tamper-proof decentralized ledger that can execute generic computer programs as smart contracts [9]. Ramanan and Nakayama designed a novel model partition and serialization method for parallel model updates. They employed the Ethereum blockchain to orchestrate the FL process, thus realizing a decentralized aggregator-free FL system [16]. Kang et al.

utilized the blockchain for secure reputation management and designed a reputation-based participant selection algorithm that improves the quality of FL [17]. The authors of [18] proposed a privacy-preserved data sharing mechanism by incorporating federated learning with the permissioned blockchain, which enhances the security of the data sharing process without a trustable center. Recently, Warnat-Herresthal et al. proposed swarm learning that leverages blockchain as the coordinator of the FL system for decentralized and confidential clinical ML [8]. The swarm learning uses the blockchain smart contract to randomly select an FL node as the aggregator, achieving confidential FL. Unlike [8], [16], [17] using blockchain for FL orchestration or data storage, we leverage the blockchain as a decentralized and trustable computing platform on which we implement the model aggregation and the incentive mechanism, thus removing the need for a trustable central server.

To encourage the data owners to participate in FL tasks, the design of incentive mechanisms is of crucial importance but confronts several challenges [19]. The study in [20] proposed a competitive model update scheme to incentivize the participants and use blockchain for fair payment. A voting mechanism was implemented on the blockchain, and participants were paid based on the number of votes on their updated models. In [21], Yu et al. designed a dynamic payoff scheme that considers both the participants' contribution and their waiting time before receiving the payments. Lyu et al. proposed a collaborative FL algorithm that utilizes the participants' reputation to achieve collaborative fairness [22]. The authors in [23] employed deep reinforcement learning to compute the optimal price for the central server and optimal training scheme for the participants as the incentive mechanism. In [7], the authors formulated the FL incentive mechanism for a social welfare maximization problem and designed a distributed optimization algorithm to solve the problem. In this work, we developed a new incentive mechanism considering participants' training time and energy cost. We formulated a Stackelberg game to derive the optimal reward scheme for incentivizing FL participants.

III. SYSTEM MODEL

A. THE DECENTRALIZED FEDERATED LEARNING ARCHITECTURE

We consider the typical industrial IoT scenario of smart manufacturing as an example application to illustrate our decentralized FL architecture. As shown in Fig. 1, each smart factory possesses a limited amount of data collected from its own manufacturing process; however, these manufacturing data usually contain privacy-sensitive data and thus cannot be shared with other manufacturers. Therefore, multiple smart factories form an FL group to learn a model for smart manufacturing (e.g., damage package detection) via the blockchain-based decentralized FL platform. The FL group consists of one smart factory as the *task publisher* and multiple smart factories as the *participants*, which are all connected to the same IoT network.

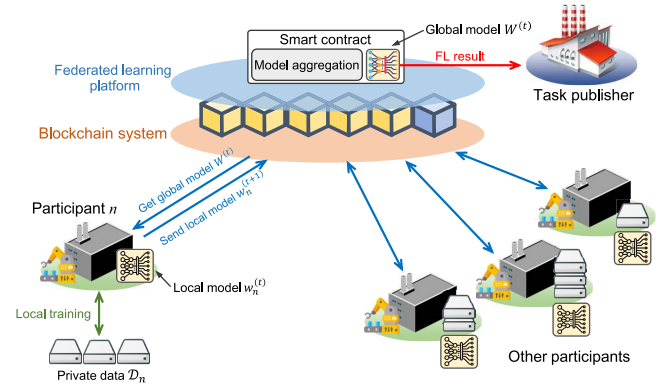


FIGURE 1. The system model of the blockchain-based decentralized federated learning architecture for smart manufacturing.

1) THE TASK PUBLISHER

The *task publisher* is the smart factory that needs to train an ML model but does not have enough training data. To this end, the task publisher can publish a learning task on the FL platform and ask other smart factories with training data to participate in the task; moreover, the task publisher also offers a reward to incentivize the smart factories that help train the FL model.

2) THE PARTICIPANTS

As illustrated in Fig. 1, the *participants* are the smart factories participating in the FL task published by the task publisher. To ease the presentation, we focus on the participants that join the same FL task. Let $n \in \mathcal{N} = \{1, 2, \dots, N\}$ denote the set of participants, where N is the total number of participants.

Each participant possesses its private training data of different quality and quantity. We denote $\mathcal{D}_n$ as participant n 's training data, where D_n is the size of $\mathcal{D}_n$ in samples. We assume supervised learning in this work; therefore, $\mathcal{D}_n$ represents a set of labeled samples such that

$$\mathcal{D}_n = \left\{ \langle \mathbf{x}_n^k, \mathbf{y}_n^k \rangle \mid k = 1, 2, \dots, D_n \right\}, \quad (1)$$

where $\langle \mathbf{x}_n^k, \mathbf{y}_n^k \rangle$ represents the k -th sample of participant n 's private data. Each labeled sample consists of two vectors: vector $\mathbf{x}_n^k$ represents the input to the ML model, e.g., photos or sensor data; vector $\mathbf{y}_n^k$ represents the expected correct output of the ML model.

B. THE FEDERATED LEARNING PROCESS

When the task publisher publishes the FL task, it initializes the global model to $W^{(0)}$, which is a two-dimensional matrix. The task publisher also uploads the test data $\mathcal{D}_T$ to the decentralized FL platform. The test data, which are unknown to the participants, are used to evaluate the ML model during the FL process. Both the global model and the test data are stored and managed by the smart contract.

The federated learning proceeds round by round iteratively, as shown in Fig. 1. In every FL round, each participant

first downloads the global model from the decentralized FL platform, then updates the model with its private data to generate a new local model, and finally uploads the local model to the FL platform. Upon receiving the local models from all the participants, the FL platform aggregates the local models to produce a new global model for the next round.

We denote t as the current number of FL round and $W^{(t)}$ as the initial global model at the t -th round. At the beginning of the t -th round, participant n first downloads the global model $W^{(t)}$, and uses its own data $\mathcal{D}_n$ to update its local model as

$$w_n^{(t+1)} = W^{(t)} - \eta \nabla F_n(W^{(t)}), \quad (2)$$

where $w_n^{(t+1)}$ is participant n 's local model at the $(t+1)$ -th round, and η represents the learning rate. The function $F_n(\cdot)$ is the loss function on the participant n 's data, which is defined as

$$F_n(w) = \frac{1}{D_n} \sum_{k=1}^{D_n} f_{n,k}(w), \quad (3)$$

where $f_{n,k}(w)$ is the loss function that evaluates the performance of model w on the k -th training sample $(\mathbf{x}_n^k, \mathbf{y}_n^k)$. The definition of the loss function depends on the specific machine learning method, such as the linear regression and the squared-support-vector machine [24]. In this work, we adopt the linear regression learning model, and thus the loss function is

$$f_{n,k}(w) = \frac{1}{2} \|\mathbf{y}_n^k - w^T \mathbf{x}_n^k\|^2. \quad (4)$$

When participant n completes its local update in (2), it sends the updated local model $w_n^{(t+1)}$ to the smart contract on the FL platform by issuing a blockchain transaction. Upon receiving the updated local model from all the participants, the smart contract automatically aggregates these local models to generate the new global model by

$$W^{(t+1)} = \frac{\sum_{n \in \mathcal{N}} D_n w_n^{(t+1)}}{\sum_{n \in \mathcal{N}} D_n}, \quad (5)$$

which is known as the federated average (FedAvg) algorithm described in [25]. In practical FL systems, the participants usually partition the training data into mini-batches of certain sizes and run several epochs of the learning function.

At the end of each FL round, the smart contract automatically evaluates the new global model $W^{(t+1)}$ with the test data $\mathcal{D}_T$. According to the task publisher's setting in the learning task, the model's quality can be measured by the loss of the model or the accuracy of the model. If the target model quality is satisfied, then the smart contract ends the FL process, returns the global model to the task publisher, and allocates the rewards among the participants; otherwise, the smart contract starts the next round of the FL process.

C. THE UNDERLYING BLOCKCHAIN SYSTEM

1) MOTIVATION

We employ blockchain to implement the decentralized FL platform for the following three considerations. First, the blockchain removes the need for a centralized coordinator and provides an open and low-cost FL platform for industrial applications. Second, the blockchain is a transparent and verifiable computing machine that can realize trusted FL and fair reward allocation. Our proposed FL platform is implemented in a smart contract, whose execution is transparent and verified by all the blockchain nodes. Third, the blockchain is also an efficient data communication network and a convenient payment tool. Furthermore, the blockchain system offers a token system that can be used to pay the FL reward online.

2) DESIGN OF THE BLOCKCHAIN SYSTEM

To support the FL application in the industrial environment, we adopt a modified version of the Ethereum [11] as the underlying blockchain system. We modified the source code of Ethereum for two goals: efficient consensus protocol and advanced smart contract functions.

The *consensus protocol* is used by the blockchain node to synchronize its local state with other nodes in the blockchain, such as PoW, PoS, DPoS, and PoA. In this work, we adopt the PoA over other consensus protocols based on the following three considerations. First, unlike the PoW protocol, the computational complexity of the PoA consensus protocol is negligible, which is feasible on industrial devices. Second, the PoA protocol provides immediate finality for the transactions, which is critical for real-time industrial applications. Third, the communication complexity of the PoA protocol is low; hence PoA is suitable for IoT networks involving massive devices.

The *smart contract* enables the blockchain users to run generic programs on the blockchain as a trusted computing machine. The smart contract is first introduced in the Ethereum blockchain project by implementing the *Ethereum virtual machine* (EVM) [11] on top of the blockchain. Solidity, a domain-specific language, is designed to program the smart contract for EVM. However, Solidity alone cannot well support the blockchain-based FL platform in this work due to several drawbacks. First, Solidity does not support floating-point number operations, which are needed by the decentralized FL platform. Second, implementing complicated arithmetic operations with Solidity, e.g., the computation of Equation (5), is inefficient and slow. Third, the smart contract written in Solidity cannot directly call the machine learning library (TensorFlow), which is needed to evaluate the model in the FL platform.

To address these issues, we implement the functions that involve floating-point numbers and TensorFlow as pre-compiled contracts written in the Go language. The pre-compiled contracts are hard-coded functions in EVM to implement some complicated but frequently used functions, such as ECDSA (digital signature) and SHA256 (hash

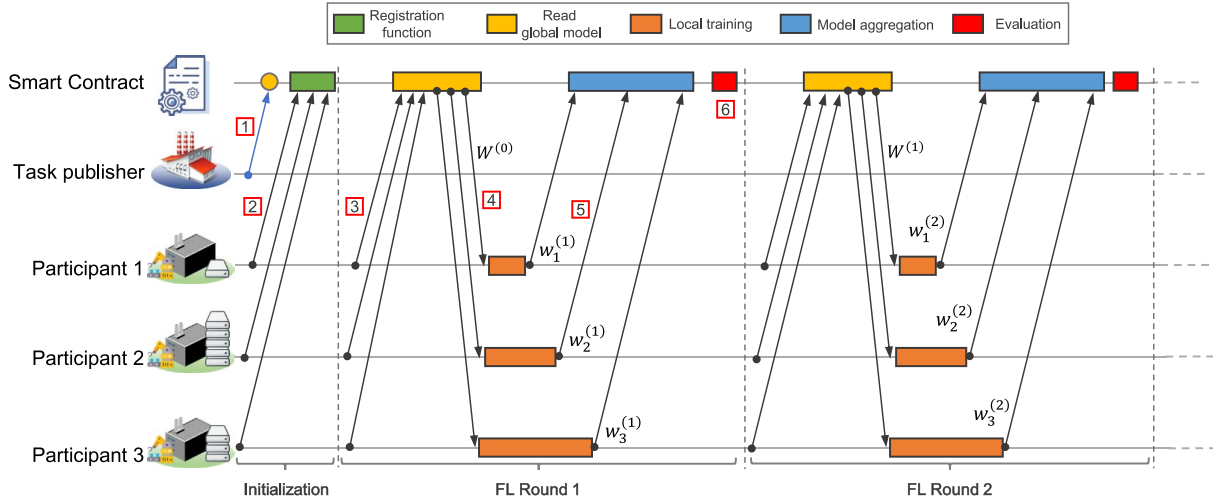


FIGURE 2. The working process of the smart contract in the decentralized FL platform. The corresponding step is marked in red square.

function) [26]. We implement the model aggregation and incentive mechanism as pre-compiled smart contracts in the Go language; also, we implement the ML-related functions in pre-compiled smart contracts by calling the TensorFlow library. These pre-compiled contracts can be called as normal functions by smart contracts written in Solidity to implement the FL platform, as discussed in Section III-C3.

3) WORKFLOW OF THE SMART CONTRACT

Fig. 2 illustrates the workflow of the smart contract that supports the decentralized FL platform. In this figure, we show an FL group of one task publisher and three participants. In the initialization phase, the task publisher first publishes the FL task and deploys the smart contract (step 1). The task publisher also needs to deposit a certain amount of tokens in the smart contract as the reward. In this phase, the smart factories that wish to participate in the FL task can call the registration function of the smart contract to become a participant (step 2). The FL process begins when enough participants have registered for the FL task. In each FL round, the participants first call the smart contract to read the current global model $W^{(0)}$ (step 3). The participants then update the global model with their private training data to produce the local models (step 4). Finally, the participants upload their local models to the smart contract for model aggregation (step 5). At the end of each FL round, the smart contract automatically evaluates the new global model (step 6). If the global model reaches the target quality, the smart contract ends the FL task and returns the latest global model to the task publisher; otherwise, a new FL round starts. In step 6, the incentive mechanism is implemented to reward the participants based on their performance in the current FL round, which will be discussed in Section IV.

D. SECURITY DISCUSSION

Compared to traditional FL systems that depend on centralized aggregators, our decentralized FL system utilizes a

blockchain as the aggregator for on-chain model aggregation. The model aggregation function is realized through a smart contract on an Ethereum-like blockchain, operating under the PoA consensus protocol. The smart contract aggregator can be safely executed as long as the conditions for PoA security are met, ensuring that the ratio of compromised validating nodes remains below the 1/3 threshold. Therefore, under the condition that the compromised validating nodes ratio stays below 1/3, our decentralized FL system offers a security level equivalent to that of traditional centralized FL systems.

IV. INCENTIVE MECHANISM DESIGN

A. THE TASK PUBLISHER AND PARTICIPANT MODELS

We consider a typical scenario, in which a single task publisher publishes one learning task in the decentralized FL platform, and N participants decide to work on the task. As discussed in Section III-B, the task publisher deposits a reward into the smart contract of the learning task to incentivize the participants. The participants are rewarded at the end of each FL round, as described in Section III-C3.

1) THE TASK PUBLISHER

When publishing the FL task on the blockchain, the task publisher deposits a reward into the smart contract to pay the participants in each FL round. The task publisher's goal is to obtain an accurate model within the shortest time. To this end, the task publisher rewards the participants based on the *contribution to the model* as well as the *time to complete the learning task*.

In each FL round, the task publisher sets a reward price π and pays the participants with rewards $\pi \frac{A_n}{t_n}$. Note that A_n denotes the participant n 's contribution to the global model, which can be measured by the difference between the loss of its local model $w_n^{(t)}$ and the loss of the global model $W^{(t)}$. We denote t_n as the time spent by participant n to complete the FL round. Therefore, the overall reward R that the task

publisher needs to pay is $R = \sum_{n \in \mathcal{N}} \pi \frac{A_n}{t_n}$. Furthermore, we also consider the completion time of the FL task as part of the task publisher's cost. We denote λ as the task publisher's sensitivity to the completion time of the task. Then, the total cost of the task publisher can be formulated as

$$\Omega_{TP} = \lambda \max_{n \in \mathcal{N}} t_n + \sum_{n \in \mathcal{N}} \pi \frac{A_n}{t_n}, \quad (6)$$

where the first term $\max_{n \in \mathcal{N}} t_n$ denotes the completion time of the FL round, since the round is completed until all the participants finish their local training.

2) THE TASK PARTICIPANT

The main cost of the participants is the power consumption for training the local model in Equation (2) using their ML processors such as CPU, GPU, FPGA, or even ASIC chips [27]. We denote f_n as the number of data samples that participant n can process per second, which measures the participant's computing speed. The participant can adjust its own computational power within the range $[F_n, \bar{F}_n]$ by changing the clock rate of its processor or by controlling the number of working cores.

For each participant, the time used for updating its local model depends on the training data size, computational power, and the FL configuration in the task. Hence, the completion time of the participant n can be modeled as $t_n = \beta_n \frac{D_n}{f_n}$, $F_n \leq f_n \leq \bar{F}_n$, where D_n is the data size, and the parameter β_n summarizes the FL configurations, such as the mini-batch size and the number of epochs per round. Therefore, the range of t_n is also bounded by $t_n \in [\underline{T}_n, \bar{T}_n]$, where $\underline{T}_n = \beta_n \frac{D_n}{\bar{F}_n}$, $\bar{T}_n = \beta_n \frac{D_n}{F_n}$.

The power consumption of the participant's ML hardware, however, is proportional to the square of its computing speed [28]. Therefore, we model the power consumption of the participant n in one FL round by

$$p_n = f_n^2 t_n = \frac{\beta_n^2 D_n^2}{t_n}, \quad (7)$$

where p_n denotes the participant n 's power consumption cost for this FL round.

By finishing the training task in each FL round, the participant n can obtain a reward of $\pi \frac{A_n}{t_n}$ as described in Section IV-A1. According to economics principles, in particular, the law of diminishing marginal utility, the participants' utility functions are chosen to be a logarithmic function. Therefore, the participant n 's utility function is $u_n = \log(1 + \frac{A_n \pi}{t_n})$. Taking the participant's power cost in (7) into consideration, the participant n 's cost is

$$\Omega_n = \gamma p_n - u_n = \frac{\gamma \beta_n^2 D_n^2}{t_n} - \log\left(1 + \frac{A_n \pi}{t_n}\right), \quad (8)$$

where γ represents the participant's sensitivity to the power consumption (e.g., the electricity price). Note that the participants can decide the value of t_n by adjusting its computing speed f_n .

B. STACKELBERG GAME AND SOLUTION METHOD

We formulate a Stackelberg game consisting of two stages to capture the interaction between the task publisher and participants. The first stage is formulated as an upper-level problem for the task publisher to optimize the reward price for its cost minimization. The second stage consists of N lower-level problems for participants to respond to the reward price and optimize their task computing time in parallel.

Upper Level: Task Publisher's Cost Minimization Problem

$$\begin{aligned} & \text{minimize } \lambda \max_{n \in \mathcal{N}} t_n + \sum_{n \in \mathcal{N}} \pi \frac{A_n}{t_n} \\ & \text{subject to } 0 \leq \pi \leq \Pi_{\max} \\ & \text{with variables: } \pi, \end{aligned}$$

where the task publisher determines the optimal reward price π for participants to minimize the waiting time $\lambda \max_{n \in \mathcal{N}} t_n$, where λ is a weight coefficient, and total reward provided to all participants $\sum_{n \in \mathcal{N}} \pi \frac{A_n}{t_n}$. Note that t_n is participants' response and a function of task publisher's reward price π .

Lower-Level: Participant n 's Cost Minimization Problem

$$\begin{aligned} & \text{minimize } \frac{\gamma \beta_n^2 D_n^2}{t_n} - \log\left(1 + \frac{A_n \pi}{t_n}\right) \\ & \text{subject to } \underline{T}_n \leq t_n \leq \bar{T}_n \\ & \text{with variables: } t_n, \end{aligned}$$

where participants receive reward price π from the task publisher and optimize their costs by determining the optimal time t_n through adjusting f_n for completing the task.

To solve the equilibrium of the formulated Stackelberg game, we use backward induction and first analyze the best response of participants given reward price π in the lower-level problems. Specifically, we verify that participant n 's cost minimization problem is strongly convex in t_n . We then derive the Karush-Kuhn-Tucker (KKT) conditions of participant n 's cost minimization problem as follows.

$$-\frac{\gamma \beta_n^2 D_n^2}{t_n^2} + \frac{\frac{A_n \pi}{t_n^2}}{1 + \frac{A_n \pi}{t_n}} - \lambda_{n,1} + \lambda_{n,2} = 0, \quad (9a)$$

$$\underline{T}_n \leq t_n \leq \bar{T}_n, \quad (9b)$$

$$\lambda_{n,1}(\underline{T}_n - t_n) = 0, \quad (9c)$$

$$\lambda_{n,2}(t_n - \bar{T}_n) = 0, \quad (9d)$$

$$\lambda_{n,1}, \lambda_{n,2} \geq 0, \quad (9e)$$

where $\lambda_{n,1}$ and $\lambda_{n,2}$ are the Lagrange multipliers associated with the Constraints (9b).

The traditional solution method is to incorporate the KKT conditions in Equation (9) into the task publisher's cost minimization problem at the upper level and jointly optimize both the task publisher's variable π and all participants' variables $\{t_n, \forall n \in \mathcal{N}\}$. However, the stationarity condition in (9) is a nonconvex constraint for the overall optimization problem, which is challenging to solve. Therefore, we seek a new solution method by leveraging the unique structure in

the task publisher's cost minimization problem and the best responses of participants.

To be more specific, we further analyze the best response t_n given the reward price π . From the KKT conditions in Equation (9), we can solve each participant's optimal response as a function of π as

$$t_n^* = \max\{\underline{T}_n, \min\{\hat{t}_n, \bar{T}_n\}\}, \quad (10)$$

where

$$\hat{t}_n = \frac{\gamma\beta_n^2 D_n^2 A_n \pi}{A_n \pi - \gamma\beta_n^2 D_n^2}, \quad (11)$$

which is a monotonically decreasing function of π . This means that participants will determine a smaller t_n under a greater reward price π . However, the reward price π is the task publisher's decision variable, and the task publisher needs to trade-off between how efficiently the FL task is completed and how much incentive as a cost to task publisher should be provided.

After analyzing the best response of participants, we examine the task publisher's cost minimization problem at the upper level. From the publisher's objective function in Equation (6), we see that the task publisher needs to trade-off the efficiency of finishing the task measured by participants' maximum time to complete the task denoted as $\max_{n \in \mathcal{N}} t_n$ and the total reward provided to participants denoted as $\sum_{n \in \mathcal{N}} \pi \frac{A_n}{t_n}$. We can further analyze the first-order and second-order derivatives of $C_1(\pi) = t_n$ and $C_2(\pi) = \pi \frac{A_n}{t_n}$ as follows.

As we discussed that the valid reward price satisfies $\pi > \frac{\gamma\beta_n^2 D_n^2}{A_n}$, we have $C_1(\pi)' < 0$ and $C_1(\pi)'' > 0$ as follows.

$$C_1(\pi)' = \begin{cases} \frac{-A_n(\gamma\beta_n^2 D_n^2)^2}{(A_n \pi - \gamma\beta_n^2 D_n^2)^2} < 0 & t_n \in (\underline{T}_n, \bar{T}_n), \\ 0 & \text{otherwise,} \end{cases} \quad (12)$$

where t_n is monotonically decreasing with the reward price π , and

$$C_1(\pi)'' = \begin{cases} \frac{2(A_n \gamma\beta_n^2 D_n^2)^2}{(A_n \pi - \gamma\beta_n^2 D_n^2)^3} > 0 & t_n \in (\underline{T}_n, \bar{T}_n), \\ 0 & \text{otherwise,} \end{cases} \quad (13)$$

where the completion time $C_1(\pi)$ is a convex function in the reward price π .

Similarly, we calculate the first-order and second-order derivatives of $C_2(\pi) = \frac{A_n \pi - \gamma\beta_n^2 D_n^2}{\gamma\beta_n^2 D_n^2}$, which is a linear function of π , and obtain

$$C_2(\pi)' = \frac{A_n}{\gamma\beta_n^2 D_n^2} > 0 \text{ and } C_2(\pi)'' = 0, \quad (14)$$

which indicate that the reward provided to participant is linearly increasing in the reward price π .

Based on the analysis in Equations (12)–(14), we prove that the task publisher's objective is a convex function of π , and its cost minimization problem is a convex optimization problem in π . To solve the optimal reward price π^* , we can use gradient descent to iteratively search for the optimal

Algorithm 1: Optimal Reward Searching Algorithm for the Task Publisher

```

1 Initialization:
2   Iteration number  $k \leftarrow 1$ ;
3   Initial reward price  $\pi(0) \leftarrow \Pi_{\max}$ ;
4   The termination threshold  $\epsilon \leftarrow 0.0001$ ;
5   Step size  $\eta \leftarrow 0.01$ ;
6 while  $|\pi(k) - \pi(k-1)| \geq \epsilon$  do
7   for  $n \in \mathcal{N}$  do
8     Participant  $n$  solves its best response  $t_n^*(k)$  by
      Equation (10) given  $\pi(k)$ ;
9     Participant  $n$  reports  $t_n^*(k)$  to the task publisher;
10  end
11  The task publisher collects  $t_n^*(k), n \in \mathcal{N}$  from all
    participants;
12  The task publisher updates the reward price by the
    gradient descent

    
$$\pi(k+1) \leftarrow \pi(k) - \eta \left( \lambda C_1^{\hat{n}}(\pi(k))' + \sum_{n \in \mathcal{N}} C_2(\pi(k))' \right),$$


    where  $\hat{n} = \arg \max_{n \in \mathcal{N}} t_n^*(k)$ ;
13   $k \leftarrow k + 1$ ;
14 end
Result: The optimal reward price  $\pi^*$ .
    
```

reward price for the task publisher interacting with all participants. The optimal reward searching algorithm for the task publisher is described in Algorithm 1.

V. SYSTEM IMPLEMENTATION AND EVALUATION

A. PROTOTYPE SYSTEM

We build a prototype system using the docker containers to evaluate the blockchain-based decentralized FL in a practical blockchain network. As shown in Fig. 3, the prototype system runs on a cluster of Dell PowerEdge servers with dual Xeon Gold 6230R CPU and 1024GB Memory, on which we create 400 docker containers in Ubuntu 18.04 OS. Specifically, we use the docker container as an FL node consisting of the federated learning software, the blockchain software, and the network simulator. We developed the blockchain software based on the open-source Ethereum client software Geth [26] written in Go language. We modified the smart contract module and EVM module of Geth to support floating-pointing operation, on-chain model aggregation, and incentive mechanism as described in Section III-C.

The network simulator is implemented on top of the Linux `tc` (traffic control) tool that can control the network parameters (latency, bandwidth, and packet loss rate) of the container's outgoing network traffic. We use a centralized network controller to configure the network parameters of all the containers. To simulate the network condition of practical industrial IoT scenario, we limit the bandwidth of each node to 1 Mbps and add a random network delay $t_d \in [0, 120]$ ms

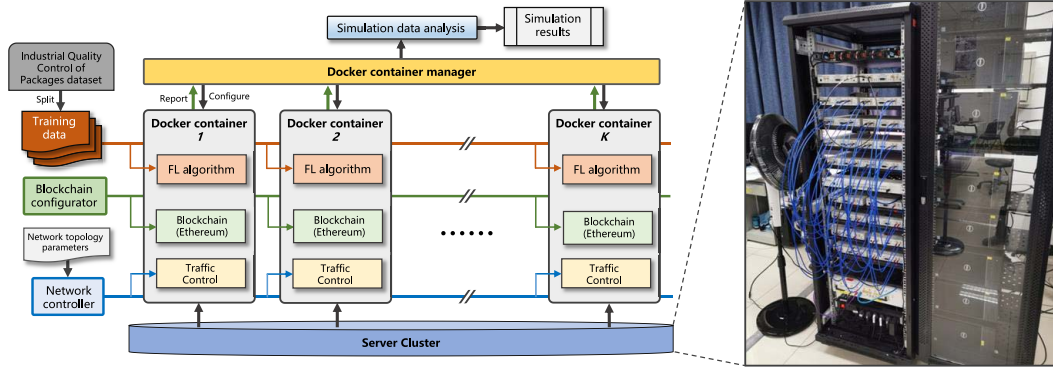


FIGURE 3. The prototype system of the blockchain-based federated learning implemented on a cluster of Dell PowerEdge servers (right).

to each transmission. As a result, this prototype system can simulate a practical industrial IoT network of 400 nodes.

We implement the local training part of the FL algorithm using the TensorFlow software pack [29]. Without loss of generality, we employ the *industrial quality control of packages* dataset from Kaggle [30] as an example FL application to evaluate the proposed method. This dataset consists of RGB images (PNG format) of product packages taken from production line. For each product package, the dataset provides two images captured by a top-view camera and a side-view camera, respectively. The resolution of the image is 960×540 and the size of one image is about 600 KB. The dataset of product packages are classified into two categories: damaged product package and intact product package. The goal of the FL system is to train an ML model that can distinguish damaged and intact product packages based on the top-view and side-view images of the product.

We mixed the top-view and side-view images to generate 50000 data samples, including 40000 samples of damaged product packages and 10000 samples of intact product packages. Each data sample contains one top-view image and one side-view image. We divide the data sample into a training dataset of 40000 samples and a test dataset of 10000 samples. Specifically, we randomly split the training dataset into the ten FL participants so that different participants occupy different amounts of training data (as their private data). In addition, part of the dataset (4000 samples) is used by the smart contract to measure the training contribution and damaged package detection accuracy of the global model. Within each docker container, we implement a double-layer neural network with TensorFlow for local model training, consisting of 7,960 parameters in total.

B. PERFORMANCE EVALUATION

1) CONVERGENCE OF THE INCENTIVE MECHANISM

As discussed in Section IV, we design the optimal reward algorithm for the task publisher in Algorithm 1, which works iteratively to search for the optimal reward price in the incentive mechanism. In our experiment, we set the target model accuracy to 93% so that the FL process ends at the 8th

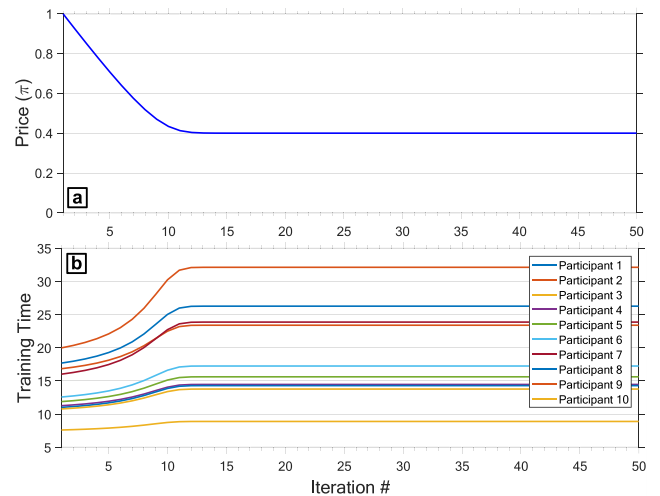


FIGURE 4. The convergence of the optimal reward price searching algorithm (Algorithm 1): (a) the convergence of the reward price; (b) the convergence of participants' decisions on their training time.

round when the global model's accuracy reaches the target. In Fig. 4, we plot the convergence of the reward price π (Fig. 4a) and the participants' training time (Fig. 4b) in the first FL round as an example. We can see that the reward price and the training times converge to their optimal values after 13 iterations. Moreover, we also observe similar results in other rounds of the FL process. These results show that the optimal reward algorithm performs well with practical application data and converges fast to the optimal solution.

2) THE TASK PUBLISHER'S REWARD PRICE AND COST

In each FL round, the task publisher's reward price is determined by Algorithm 1, and its cost is calculated by Equation (6). Fig. 5 shows the task publisher's cost (blue bars) and reward price (red line) in each FL round. We observe that the reward price drops gradually from 0.41 to 0.26 as the FL algorithm proceeds. The rationale behind this phenomenon is that the participants' contributions to the global model decrease as the number of FL rounds increases. However, the task publisher's cost does not decrease rapidly with the price, because the participants' training time increases when the reward price falls.

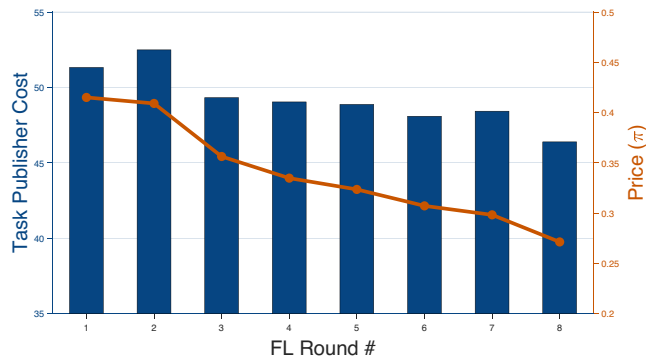


FIGURE 5. The task publisher's total cost (blue bars) and its optimal reward price (red line) in the 8 rounds of federated learning.

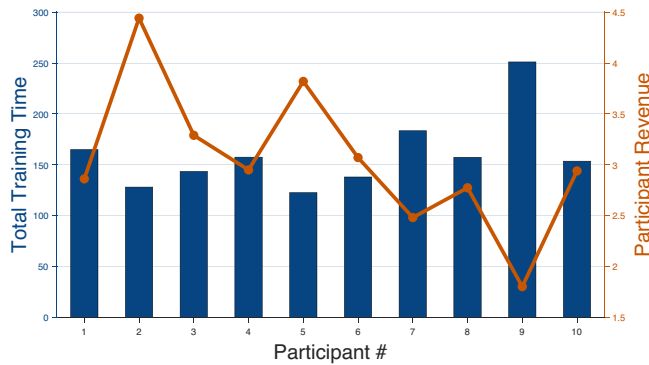


FIGURE 6. The participants' total training time (blue bars) and total revenue (red line) in the FL task.

3) THE TASK PARTICIPANTS' TRAINING TIME AND REVENUE

To evaluate the performance of the incentive mechanism, we plot the total training time and overall revenue of ten participants during the FL task in Fig. 6. The participant's total training time is the summation of its training time used in all the eight FL rounds, and the participant's revenue is the inverse of its cost defined in Equation (8). In Fig. 6, we can see that the participant's overall revenue is inversely proportional to its training time, which is consistent with the payment scheme. We also notice that the participants' revenue is proportional to their contributions to the global model. For example, although participant 2's energy cost is much higher than that of participant 9, participant 2 makes more revenue because participant 2 contributes more to the global model than participant 9. The experiment results in Fig. 6 show that the proposed optimal reward algorithm is feasible in practical FL applications and can provide fair and effective incentives to the FL participants.

VI. CONCLUSION

This paper presented a blockchain-based decentralized federated learning (FL) architecture for industrial IoT. This architecture enables the users to form a trustable and decentralized FL group without a centralized server or a third-party FL platform. We implemented the model

aggregation process (FedAvg) and the incentive mechanism using the pre-compiled smart contract on the blockchain. The incentive mechanism is designed based on a two-stage Stackelberg game that achieves optimal reward for the task publisher and participants. We built a prototype system of the proposed FL architecture and evaluated it with a damaged package detection application. Experiment results showed that the FL system could achieve a detection accuracy of 93% in 8 rounds, and the incentive mechanism converged to the optimal rewarding scheme in 13 iterations. Our work verifies that a fully decentralized FL platform without a central server is feasible in a practical industrial IoT scenario.

For future work, there are several promising directions to extend the proposed blockchain-based federated learning architecture. One important direction is extending the current incentive mechanism to handle more complex scenarios, such as dynamically changing numbers of participants, different contribution evaluation metrics, or multiple task publishers. The current Stackelberg game formulation could be generalized. Another valuable research direction is integrating additional privacy-preserving techniques into the system, such as differential privacy or secure multi-party computation, to provide even stronger privacy guarantees for the participants' local data and models.

REFERENCES

- [1] L. U. Khan, W. Saad, Z. Han, E. Hossain, and C. S. Hong, "Federated learning for Internet of Things: Recent advances, taxonomy, and open challenges," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 3, pp. 1759–1799, 3rd Quart., 2021.
- [2] M. Ghahramani, Y. Qiao, M. C. Zhou, A. O'Hagan, and J. Sweeney, "AI-based modeling and data-driven evaluation for smart manufacturing processes," *IEEE/CAA J. Automatica Sinica*, vol. 7, no. 4, pp. 1026–1037, Jul. 2020.
- [3] M. I. Jordan and T. M. Mitchell, "Machine learning: Trends, perspectives, and prospects," *Science*, vol. 349, no. 6245, pp. 255–260, 2015.
- [4] A. R. Sani, M. U. Hassan, and J. Chen, "Privacy preserving machine learning for electric vehicles: A survey," 2022, *arXiv:2205.08462*.
- [5] C. Tikkinen-Piri, A. Rohunen, and J. Markkula, "EU general data protection regulation: Changes and implications for personal data collecting companies," *Comput. Law Secur. Rev.*, vol. 34, no. 1, pp. 134–153, 2018.
- [6] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated machine learning: Concept and applications," *ACM Trans. Intell. Syst. Technol.*, vol. 10, no. 2, pp. 1–19, 2019.
- [7] M. Tang and V. W. S. Wong, "An incentive mechanism for cross-silo federated learning: A public goods perspective," in *Proc. IEEE Conf. Comput. Commun.*, 2021, pp. 1–10.
- [8] S. Warmat-Herresthal et al., "Swarm learning for decentralized and confidential clinical machine learning," *Nature*, vol. 594, no. 7862, pp. 265–270, 2021.
- [9] T. T. A. Dinh, R. Liu, M. Zhang, G. Chen, B. C. Ooi, and J. Wang, "Untangling blockchain: A data processing view of blockchain systems," *IEEE Trans. Knowl. Data Eng.*, vol. 30, no. 7, pp. 1366–1385, Jul. 2018.
- [10] C. Natoli, J. Yu, V. Gramoli, and P. Esteves-Verissimo, "Deconstructing blockchains: A comprehensive survey on consensus, membership and structure," 2019, *arXiv:1908.08316*.
- [11] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," Ethereum, Zug, Switzerland, Yellow Paper, 2019. [Online]. Available: <https://ethereum.github.io/yellowpaper/paper.pdf>
- [12] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Artif. Intell. Statist.*, 2017, pp. 1273–1282.

- [13] R. C. Geyer, T. Klein, and M. Nabi, "Differentially private federated learning: A client level perspective," 2017, *arXiv:1712.07557*.
- [14] K. Bonawitz et al., "Practical secure aggregation for privacy-preserving machine learning," in *Proc. ACM SIGSAC*, 2017, pp. 1175–1191.
- [15] M. Hao, H. Li, G. Xu, S. Liu, and H. Yang, "Towards efficient and privacy-preserving federated deep learning," in *Proc. IEEE ICC*, 2019, pp. 1–6.
- [16] P. Ramanan and K. Nakayama, "BAFFLE: Blockchain based aggregator free federated learning," in *Proc. IEEE Int. Conf. Blockchain*, 2020, pp. 72–81.
- [17] J. Kang, Z. Xiong, D. Niyato, S. Xie, and J. Zhang, "Incentive mechanism for reliable federated learning: A joint optimization approach to combining reputation and contract theory," *IEEE Internet Things J.*, vol. 6, no. 6, pp. 10700–10714, Dec. 2019.
- [18] Y. Lu, X. Huang, Y. Dai, S. Maharjan, and Y. Zhang, "Blockchain and federated learning for privacy-preserved data sharing in Industrial IoT," *IEEE Trans. Ind. Informat.*, vol. 16, no. 6, pp. 4177–4186, Jun. 2020.
- [19] Y. Zhan, J. Zhang, Z. Hong, L. Wu, P. Li, and S. Guo, "A survey of incentive mechanism design for federated learning," *IEEE Trans. Emerg. Topics Comput.*, vol. 10, no. 2, pp. 1035–1044, Apr./Jun. 2022.
- [20] K. Toyoda and A. N. Zhang, "Mechanism design for an incentive-aware blockchain-enabled federated learning platform," in *Proc. IEEE Int. Conf. Big Data*, 2019, pp. 395–403.
- [21] H. Yu et al., "A sustainable incentive scheme for federated learning," *IEEE Intell. Syst.*, vol. 35, no. 4, pp. 58–69, Jul./Aug. 2020.
- [22] L. Lyu, X. Xu, Q. Wang, and H. Yu, "Collaborative fairness in federated learning," in *Federated Learning*. Cham, Switzerland: Springer, 2020, pp. 189–204.
- [23] Y. Zhan, P. Li, Z. Qu, D. Zeng, and S. Guo, "A learning-based incentive mechanism for federated learning," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 6360–6368, Jul. 2020.
- [24] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2017.
- [25] J. Konečný, H. B. McMahan, F. X. Yu, P. Richtárik, A. T. Suresh, and D. Bacon, "Federated learning: Strategies for improving communication efficiency," 2016, *arXiv:1610.05492*.
- [26] (Ethereum, Zug, Switzerland). *Go-Ethereum Pre-Compiled Contracts, Version 1.7.0*. Accessed: Dec. 1, 2020. [Online]. Available: <https://github.com/ethereum/go-ethereum/blob/master/core/vm/contracts.go>
- [27] Y. LeCun, "Deep learning hardware: Past, present, and future," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, 2019, pp. 12–19.
- [28] I. Takouna, W. Dawoud, and C. Meinel, "Accurate multicore processor power models for power-aware resource management," in *Proc. IEEE 9th Int. Conf. Dependable, Auton. Secure Comput.*, 2011, pp. 419–426.
- [29] M. Abadi et al. "TensorFlow: Large-scale machine learning on heterogeneous systems." 2015. [Online]. Available: <http://tensorflow.org/>
- [30] "Industrial quality control of packages." Kaggle. Accessed: Feb. 1, 2022. [Online]. Available: <https://www.kaggle.com/datasets/christianvorhemus/industrial-quality-control-of-packages>